Lesson 12 · Cloud-Native .NET on AWS & Modernization

Ship MotorQuote to AWS Lambda and ECS Fargate from one VB rating library — Annotations, SnapStart, SDK container images, Terraform, OIDC, OpenTelemetry — then triage a .NET Framework estate

Contents

1 · Why this lesson — MotorQuote meets the AWS estate you already run 2
- 1.1 · The numbers to leave this lesson with 2

2 · Choosing a hosting target 2
- 2.1 · Six ways to host MotorQuote on AWS 3
- 2.2 · A decision flow for one workload 4
- 2.3 · Hosting targets against the questions that decide them 4
- 2.4 · Concept map — Python, JVM and AWS habits → .NET on AWS 4

3 · .NET on Lambda — handlers, Annotations and SnapStart 5
- 3.1 · POST /quotes — a Python Powertools handler and its Annotations twin 6
- 3.2 · Startup — what runs once per execution environment 6
- 3.3 · One cold request, then a warm one 7
- 3.4 · The same handler without Annotations — C# and Visual Basic 7
- 3.5 · SnapStart hooks — warm before the snapshot, renew after restore 8
- 3.6 · .NET runtimes on Lambda 8
- 3.7 · Handler code lines — Annotations vs by hand 9
- 3.8 · Deployment package size 9
- 3.9 · Testing the class Lambda actually calls 9

4 · Containers without a Dockerfile 9
- 4.1 · The project file is the Dockerfile 10
- 4.2 · Two hosting targets, one shutdown budget 10
- 4.3 · Liveness and readiness without a shell 11
- 4.4 · An ECS task's life, from pull to SIGKILL 11
- 4.5 · What you should see — two images and an index 11
- 4.6 · What is inside the image, per architecture 12
- 4.7 · The image configuration the SDK wrote 12

5 · Infrastructure and delivery — Terraform, OIDC, CDK and Aspire 12
- 5.1 · From push to running, with no stored keys 13
- 5.2 · The workflow — test, authenticate, package 13
- 5.3 · The workflow — push the image, apply the plan 14
- 5.4 · Terraform — the function, SnapStart and a live alias 14
- 5.5 · Terraform — the Fargate task and its target group 15
- 5.6 · Three ways to describe the same infrastructure 15

6 · Observability — OpenTelemetry on .NET's own APIs 15
- 6.1 · Wiring traces, metrics and logs 16
- 6.2 · One request: a span, a counter and a structured log 16
- 6.3 · Where each signal goes 17
- 6.4 · Asserting on spans in a test 17

7 · Modernising Windows and IIS estates 17
- 7.1 · Porting map — .NET Framework on IIS → .NET 10 18
- 7.2 · The triage rules — one signal, one arm 18
- 7.3 · What you should see — sixteen applications, seven Rs 19
- 7.4 · Applications per migration strategy 19
- 7.5 · Applications per target platform 19
- 7.6 · Strangling one IIS application behind YARP 20
- 7.7 · Modernisation tools as of September 2026 20

8 · Hands-on — and what to learn after the track 20
- 8.1 · Commands 21
- 8.2 · What you should see 21
- 8.3 · Code lines per lesson-12 project 21

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples.

1 · Why this lesson — MotorQuote meets the AWS estate you already run

You have already shipped the hard parts of this lesson, in other languages. Python Lambda functions tuned to ~175 ms cold starts and 55 KB packages, services on ECS Fargate, Terraform that provisions 1,000–2,000+ resources in under three minutes, GitHub Actions pipelines, and the Windows (IIS) rows of an 80+ application migration inventory. On .NET the architecture carries over; what changes is how a function is found and started, how an image is built, which API emits telemetry, and what the legacy you have to move is made of.

The samples take MotorQuote to AWS twice, from one Visual Basic rating library. `L12.RatingVb` is called by a C# Lambda function written with Lambda Annotations, by a hand-written VB Lambda handler, and by an ASP.NET Core API that publishes itself as a container image without a Dockerfile. `infra/main.tf` and `ci/deploy.yml` describe how both would ship; tests prove the handlers agree. Every premium comes from the one MotorQuote tariff the whole track prices with — **illustrative, not a real tariff** — and three or more claims in five years decline the quote: HTTP 422 on every host, not a crash. The request and stored quote are flat wire types (plain `decimal`, string id), not the `Money` and `QuoteId` structs of lesson 03 (Types, OOP & Generics), because they cross JSON and storage.

Then it turns to the estate you would actually be asked to move. A triage console sorts an illustrative 16-application .NET Framework inventory into the AWS 7 Rs and a target platform, and §7 maps every Framework API family to what replaces it on .NET 10.

This is the last lesson, so it leans on the others. Hosting, DI and health checks come from lesson 08 (ASP.NET Core Web APIs), tests from lesson 10 (Testing, TDD & 100% Coverage), identity and secrets from lesson 11 (Security & Identity with Entra ID) and publish modes from lesson 01 (The .NET Platform Map).

1.1 · The numbers to leave this lesson with

DOTNET10 ON LAMBDA	DOTNET8 ON LAMBDA	LAMBDA ZIP, C# + DI	APP LAYER IN THE IMAGE	ESTATE TRIAGE
to Nov 2028	**50 days**	**809 KB**	**0.55 MB**	**16 apps**
managed runtime · deprecation 14 Nov 2028 · verified	deprecation 10 Nov 2026 · counted from 2026-09-21	VB raw handler 78 KB · measured	of 55.7 MB linux-x64 · measured	8 replatform · 3 refactor · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured Python cloud architecture legacy

i Before you start

1. **Earlier lessons this one builds on:** lesson 01 (The .NET Platform Map) (publish modes, Native AOT), lesson 06 (VB.NET for Legacy Estates) (VB migration strategy), lesson 08 (ASP.NET Core Web APIs) (hosting, health checks), lesson 10 (Testing, TDD & 100% Coverage) (xUnit, `WebApplicationFactory`) and lesson 11 (Security & Identity with Entra ID) (identity, secrets).
2. **You will build 7 projects:** `L12.RatingVb` (VB library), `L12.QuoteLambda` + `.Tests` (C# Lambda), `L12.QuoteLambdaVb` (VB Lambda), `L12.QuoteContainer` + `.Tests` (container API) and `L12.EstateTriage` (console). `infra/main.tf` and `ci/deploy.yml` are read into the PDF and never applied.
3. **No AWS account, Docker or Terraform is needed to run anything.** Handlers are tested in process with `Amazon.Lambda.TestUtilities`; the API through `WebApplicationFactory`. Publishing an image archive needs internet access to mcr.microsoft.com but no Docker daemon.
4. Figures are marked • Verified (dated, linked), • Measured (computed from the samples or captured from a real run on the build machine on 2026-09-20) or • Estimate. No cold-start latency is quoted: none was measured.

2 · Choosing a hosting target

Choose the target from the workload's shape, not from the language: .NET runs on every AWS compute option you already use. Lambda has had a managed `dotnet10` runtime since 8 January 2026, on Amazon Linux 2023 for x86_64 and arm64 (.NET 10 runtime now available in AWS Lambda). ECS Fargate runs the same code as a Linux container. EC2 Windows with IIS — or a Windows container on ECS — keeps running what cannot leave .NET Framework yet. The decision inputs are the ones you used for Java and Python: traffic shape, latency budget, request length, statefulness and legacy dependencies.

Cold start is where .NET on Lambda differs from Python, and there are three answers. A framework-dependent zip loads assemblies and JIT-compiles methods on first use. SnapStart snapshots an initialised environment and restores copies — billed for .NET, unlike Java. Native AOT removes the JIT but must be compiled on Amazon Linux 2023 and needs source-generated serialisation. The rubric in 2.3 ranks these as a judgement; this lesson quotes no latency, because none was measured.

Lambda Managed Instances change the concurrency contract. Functions run on Amazon EC2 instances with Lambda's operating model, and one .NET process serves concurrent requests as Tasks — 32 per vCPU by default (.NET runtime for Lambda Managed Instances). Every piece of state a handler shares must be thread-safe, which is why the sample store is a `ConcurrentDictionary`.

2.1 · Six ways to host MotorQuote on AWS

hosting view

1 Lambda — managed dotnet10 runtime

cloud · Python · • Same idea

Upload a framework-dependent zip; Lambda supplies .NET 10 on Amazon Linux 2023, x86_64 or arm64.

YOU KNOW	Your Python Lambdas — same event model, same INIT phase, same 15-minute limit
PACKAGE	IL + dependencies: 809 KB zipped with a DI host, 78 KB without (3.8)
START-UP	Assemblies load and methods JIT-compile on first use — the cold start to manage
PICK WHEN	Spiky or event-driven work; HTTP API integrations time out at 30 s (HTTP API quotas)
SUPPORT	Deprecation 14 Nov 2028 (Lambda runtimes)

2 Lambda + SnapStart

cloud · • Different

Lambda initialises a published version once, snapshots memory and disk, and resumes copies.

NEEDS	dotnet8 or later · published versions or aliases · Annotations 1.6.0+
COST	For .NET: snapshot caching and each restore are billed — free only for Java
NOT WITH	Provisioned concurrency, EFS, ephemeral storage over 512 MB, OS-only runtimes
TRAP	Anything unique made during INIT is cloned into every copy (Lambda SnapStart)

3 Lambda + Native AOT

cloud · • Same idea

Compile the function to a native executable: no JIT at start-up.

YOU KNOW	GraalVM native-image on a custom runtime
BUILD	On Amazon Linux 2023 for the target architecture — the Lambda CLI uses a build container
RUNTIME	Managed runtime, or OS-only provided.al2023 (Introducing the .NET 8 runtime for AWS Lambda)
NEEDS	Source-generated JSON, trim-safe libraries, a test on Lambda (Compile .NET Lambda code to a native runtime format)

2.1 · Six ways to host MotorQuote on AWS (continued)

hosting view

4 ECS Fargate — container

cloud · architecture · • Renamed

An ASP.NET Core API in a Linux container behind a load balancer.

YOU KNOW	Spring Boot or NestJS on Fargate
IMAGE	`dotnet publish -t:PublishContainer`: chiseled, UID 1654, port 8080 (§4)
STOP	SIGTERM, then SIGKILL after stopTimeout — 30 s by default, 120 s at most
PICK WHEN	Steady traffic, long requests or connections, background work, no cold starts

5 EC2 Windows + IIS

legacy · • Different

Where .NET Framework, COM and GAC dependencies keep running while you port them.

RUNS	ASP.NET 4.x unchanged; ASP.NET Core through the Hosting Bundle's ASP.NET Core Module
DEFAULT	In-process hosting in w3wp.exe, which cannot load a single-file app (Host ASP.NET Core on Windows with IIS)
OR	An ECS Windows container; Fargate has no gMSA, EFS or EBS (Windows on Fargate)
PICK WHEN	Rehost first, then port — the 7 R triage in §7; you own OS patching

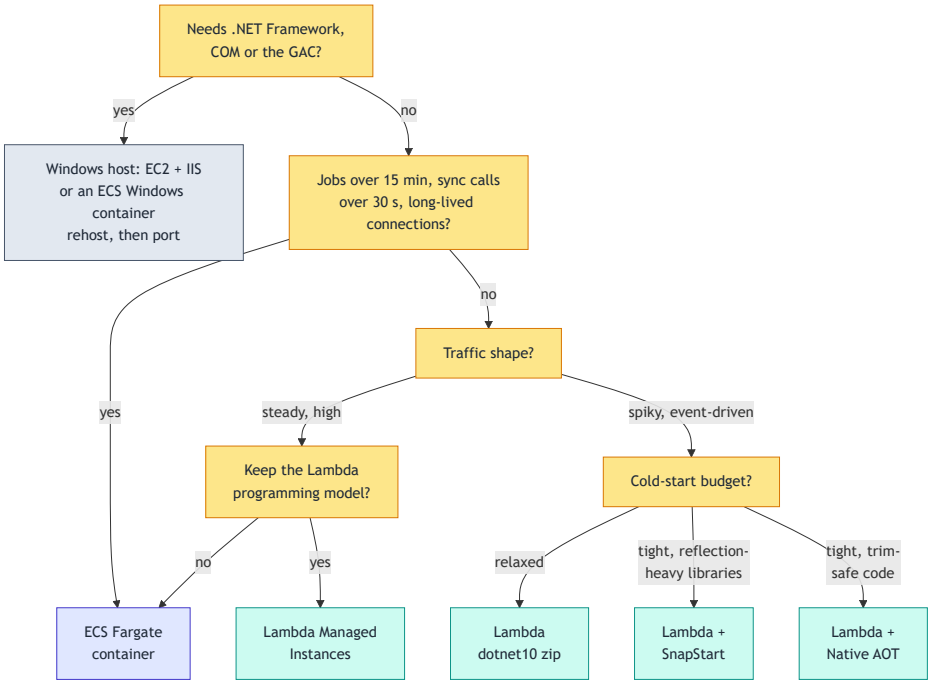
6 Lambda Managed Instances

cloud · • Trap

Lambda functions on Amazon EC2 instances, keeping Lambda's operating model; .NET 10 is supported.

MODEL	One .NET process per environment; concurrent requests run as Tasks
DEFAULT	32 concurrent requests per vCPU; Amazon.Lambda.Core 2.7.1 or later
TRAP	The handler is shared: every mutable field, static and collection must be thread-safe
GAP	Powertools and ADOT .NET instrumentation are not supported yet (.NET runtime for Lambda Managed Instances)

2.2 · A decision flow for one workload



amber = question · slate = legacy target · teal = Lambda options · indigo = container · 15 min is Lambda's maximum timeout, 30 s the HTTP API integration timeout

2.3 · Hosting targets against the questions that decide them

	Lambda	SnapStart	AOT	Fargate	EC2 IIS
Pay nothing when idle	good	some	good	poor	poor
Low cold-start latency	poor	some	some	good	good
Long requests, connections	poor	poor	poor	good	good
Reflection-heavy libraries	good	good	poor	good	good
.NET Framework / System.Web	poor	poor	poor	some	good
No OS or image patching	good	good	some	some	poor

good = strength · some = with a cost or caveat · poor = weakness · a rubric, not a benchmark

> **No column wins every row, which is why an estate ends up on several targets.** SnapStart's "some" on idle cost is its caching charge; AOT's "some" on patching is that the runtime is compiled into your binary, so a .NET patch reaches it only when you rebuild; Fargate's is the base image you rebuild. Only a Windows host runs System.Web: EC2 with IIS, or a Windows container on ECS — on Fargate with limits, hence "some" (Windows on Fargate). §7 is about making that row irrelevant. • **Estimate**

2.4 · Concept map — Python, JVM and AWS habits → .NET on AWS

You already know	In .NET	Kind	Watch for
Python handler `def handler(event, context)`	Class-library handler `ASSEMBLY::TYPE::METHOD`	• Different	Lambda finds a method on a .NET type; with Annotations the type is the generated class (5.4)
Powertools `APIGatewayHttpResolver` routes	Lambda Annotations `[LambdaFunction]` + `[HttpApi]`	• Renamed	A source generator writes the real handler; the VB sample writes its own
Module-scope clients, created once	Handler constructor · `[LambdaStartup]`	• Same idea	The Annotations function class is a singleton: no per-request state in its fields

You already know	In .NET	Kind	Watch for
SnapStart for Java — no extra charge	SnapStart for .NET 8 and later	• Trap	Billed for caching and restores; uniqueness is cloned; not on OS-only runtimes
GraalVM native-image	Native AOT on Lambda	• Same idea	Compile on Amazon Linux 2023; source-generated JSON; treat trim warnings as bugs
A thin zipped Python package	Framework-dependent `dotnet publish` zip	• Different	The generic host adds 31 files: 809 KB vs 78 KB (3.8)
Dockerfile + `docker build`	`dotnet publish -t:PublishContainer`	• Different	No `RUN`; the base comes from `ContainerFamily`; no daemon for an archive
`EXPOSE 80`, running as root	Port 8080, user app (UID 1654)	• Trap	Default since .NET 8 images — a port mapping or target group on 80 gets no answer
Actuator liveness and readiness	`MapHealthChecks` with tags	• Renamed	Chiseled images have no shell: probe from the load balancer, not `CMD-SHELL`
`server.shutdown=graceful`	`HostOptions.ShutdownTimeout` (30 s)	• Renamed	Keep it below the ECS `stopTimeout` (30 s default, 120 s max)
OpenTelemetry Java agent · Micrometer	`ActivitySource` · `Meter` · `ILogger` + OpenTelemetry SDK	• Renamed	The API is in the base library; an `Activity` is the span
X-Ray SDK + daemon	OpenTelemetry + ADOT collector or CloudWatch agent	• Different	X-Ray SDKs and daemon in maintenance mode since 25 Feb 2026
Terraform · GitHub Actions OIDC	The same tools	• Same idea	Provider builds older than the dotnet10 launch reject the runtime value
Java EE on WebSphere → Spring Boot	ASP.NET on IIS → ASP.NET Core	• Different	`System.Web` is gone: migrate route by route behind YARP (7.6)

3 · .NET on Lambda — handlers, Annotations and SnapStart

A .NET Lambda function is a method that Lambda finds by name. In the class-library model all samples use, the handler string is `ASSEMBLY::TYPE::METHOD`: Lambda loads the assembly into the managed runtime, constructs the type during INIT and calls the method per event. The alternative executable-assembly model starts the runtime client itself with `LambdaBootstrapBuilder`; .NET 10 file-based functions (one `.cs` file, no project) use it and default to Native AOT (Define Lambda function handler in C#).

Lambda Annotations is the .NET counterpart of Powertools' event handler — and it is a source generator. You write a normal method with `[LambdaFunction]` and `[HttpApi]` and constructor-injected services. At compile time it writes `QuoteFunctions_CreateQuote_Generated`, whose constructor runs `Startup`, registers the function class as a singleton and builds the container during INIT; per request it creates a DI scope, binds the HTTP API body and returns 400 when the body cannot be read. It also keeps `serverless.template` in step. AWS documents Annotations for C# functions, so the VB project writes its handler by hand (3.4).

SnapStart moves INIT from the first request to publish time. Lambda initialises a published version, snapshots it, and resumes new environments from the snapshot. What you give up: anything unique created during INIT is identical in every copy, connections opened during INIT may be stale, and for .NET you pay for caching and each restore (Lambda SnapStart). Hooks run code just before the snapshot and just after a restore; they need Amazon.Lambda.Core 2.5.0 or later (SnapStart runtime hooks for .NET).

3.1 · POST /quotes — a Python Powertools handler and its Annotations twin

Python — for comparison Powertools for AWS Lambda (Python)

```python
from aws_lambda_powertools import Logger
from aws_lambda_powertools.event_handler import (
    APIGatewayHttpResolver, Response)

logger, app = Logger(), APIGatewayHttpResolver()
store = InMemoryQuoteStore()  # module scope: once

@app.post("/quotes")
def create_quote():
    req = app.current_event.json_body
    coverage = COVERAGE.get(req["coverage"].lower())
    if coverage is None:
        return Response(400, "application/json",
                        '"unknown coverage"')
    saved = store.save(quote(coverage, req))
    logger.info("Quoted", extra={"quote_id": saved.id})
    location = f"/quotes/{saved.id}"
    return Response(201, "application/json", saved.to_json(),
                    headers={"Location": location})

def handler(event, context):  # handler: app.handler
    return app.resolve(event, context)
```

C# samples/L12.QuoteLambda/QuoteFunctions.cs · #region annotatio…

```csharp
[LambdaFunction(MemorySize = 1024, Timeout = 10)]
[HttpApi(LambdaHttpMethod.Post, "/quotes")]
public IHttpResult CreateQuote(
    [FromBody] QuoteRequest request, ILambdaContext context)
{
    if (!Enum.TryParse(request.Coverage, true,
            out CoverageClass coverage))
        return HttpResults.BadRequest("unknown coverage");

    PremiumBreakdown p;
    try
    {
        p = Rating.Quote(coverage, request.SumInsured,
            request.DriverAge, request.LicenceYears,
            request.ClaimsLast5Years, request.Commercial,
            request.EngineCc);
    }
    catch (QuoteDeclinedException declined)
    {
        return HttpResults.NewResult(
            HttpStatusCode.UnprocessableEntity,
            declined.Message);
    }
    var quote = store.Save(p);

    context.Logger.LogInformation("Quoted {QuoteId}: {Total}",
        quote.QuoteId, p.Total);
    return HttpResults.Created(
        $"/quotes/{quote.QuoteId}", quote);
}
```

Same shape, different mechanism. Python routes at run time inside `app.resolve`; Annotations routes at compile time, so a C# function is one route per Lambda function and the binding code is generated, not reflected. `[FromBody]` replaces `json_body`, `IHttpResult` replaces `Response`, and `context.Logger` takes a message template whose `{QuoteId}` becomes a JSON field when the function's log format is JSON (Log and monitor C# Lambda functions). Neither handler touches rates: both call a rating module — in .NET, the VB library — and the C# one turns its `QuoteDeclinedException` (three or more claims) into a 422, which the Python sketch leaves to its assumed helpers. **Annotations does not replace Powertools; the two are complementary.** Annotations takes over the routing, binding and DI half of Python's event handler; Powertools for AWS Lambda (.NET) has no API Gateway router of its own — its Event Handler covers AppSync Events and Bedrock Agent functions — and stays what you add for logging, metrics, tracing, idempotency, batch processing and parameters (Powertools for AWS Lambda (.NET)).

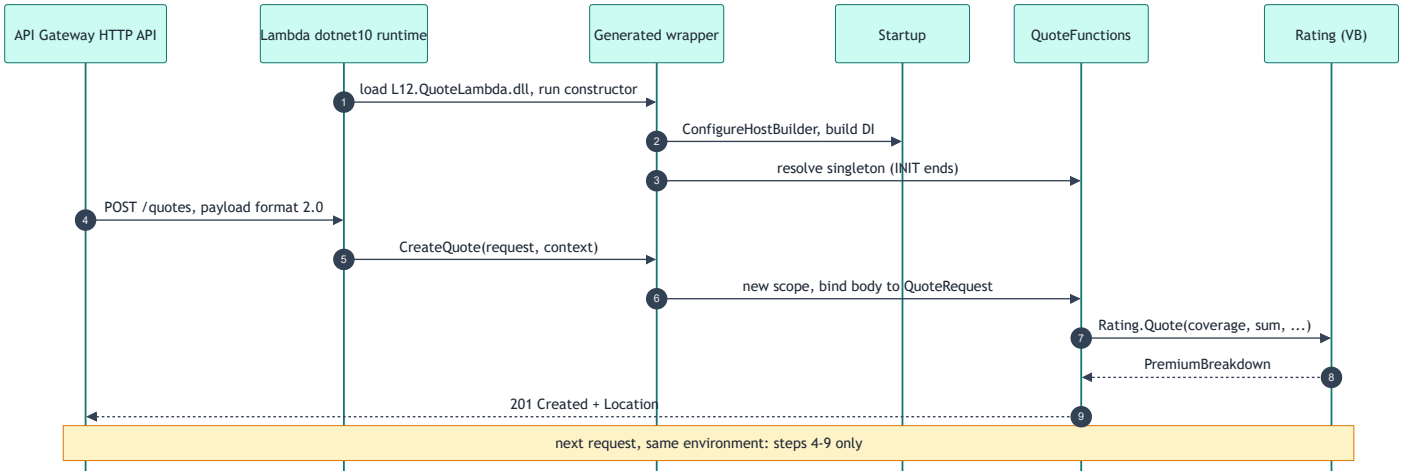
3.2 · Startup — what runs once per execution environment

C# samples/L12.QuoteLambda/Startup.cs · #region startup

```csharp
[LambdaStartup]
public sealed class Startup
{
    // Runs once per execution environment (Lambda's init phase), not once per request.
    public HostApplicationBuilder ConfigureHostBuilder()
    {
        var builder = new HostApplicationBuilder();
        builder.Services.AddSingleton<IQuoteStore, InMemoryQuoteStore>();
        SnapStart.RegisterHooks();
        return builder;
    }
}
```

This is Lambda's INIT phase, written as DI. The generated wrapper's constructor calls `ConfigureHostBuilder`, adds `QuoteFunctions` as a singleton and resolves it eagerly, so construction time shows up in the INIT report rather than the first request. A singleton store survives between invocations of one environment only; Lambda runs many environments in parallel and recycles them, which is why production state belongs in DynamoDB (lesson 09 (Data Access with EF Core) covers data access).

3.3 · One cold request, then a warm one



INIT runs once per execution environment; a warm request skips steps 1–3 · with SnapStart, steps 1–3 happen when the version is published and a restore replaces them

3.4 · The same handler without Annotations — C# and Visual Basic

C# `samples/L12.QuoteLambda/RawQuoteHandler.cs · #region raw-cs`

```
static readonly JsonSerializerOptions Json =
    JsonSerializerOptions.Web;

public APIGatewayHttpApiV2ProxyResponse FunctionHandler(
    APIGatewayHttpApiV2ProxyRequest request,
    ILambdaContext context)
{
    var input = JsonSerializer.Deserialize<QuoteRequest>(
        request.Body ?? "{}", Json);
    if (input is null ||
        !Enum.TryParse(input.Coverage, true,
            out CoverageClass coverage))
        return Respond(400, "\"unknown coverage\"");

    try
    {
        var p = Rating.Quote(coverage, input.SumInsured,
            input.DriverAge, input.LicenceYears,
            input.ClaimsLast5Years, input.Commercial,
            input.EngineCc);
        return Respond(201,
            JsonSerializer.Serialize(p, Json));
    }
    catch (QuoteDeclinedException declined)
    {
        return Respond(422, JsonSerializer.Serialize(
            declined.Message, Json));
    }
}

static APIGatewayHttpApiV2ProxyResponse Respond(
    int status, string body) => new()
{
    StatusCode = status,
    Body = body,
    Headers = new Dictionary<string, string>
        { ["Content-Type"] = "application/json" },
};
```

Visual Basic `samples/L12.QuoteLambdaVb/QuoteHandler.vb · #region …`

```
Private Shared ReadOnly Json As JsonSerializerOptions =
    JsonSerializerOptions.Web

Public Function FunctionHandler(
        request As APIGatewayHttpApiV2ProxyRequest,
        context As ILambdaContext) _
        As APIGatewayHttpApiV2ProxyResponse
    Dim input = JsonSerializer.Deserialize(Of QuoteInput)(
        If(request.Body, "{}"), Json)
    Dim coverage As CoverageClass
    If input Is Nothing OrElse Not [Enum].TryParse(
            input.Coverage, True, coverage) Then
        Return Respond(400, """unknown coverage""")
    End If

    Try
        Dim p = Rating.Quote(coverage, input.SumInsured,
            input.DriverAge, input.LicenceYears,
            input.ClaimsLast5Years, input.Commercial,
            input.EngineCc)
        Return Respond(201, JsonSerializer.Serialize(p, Json))
    Catch declined As QuoteDeclinedException
        Return Respond(422, JsonSerializer.Serialize(
            declined.Message, Json))
    End Try
End Function

Private Shared Function Respond(status As Integer,
        body As String) As APIGatewayHttpApiV2ProxyResponse
    Return New APIGatewayHttpApiV2ProxyResponse With {
        .StatusCode = status,
        .Body = body,
        .Headers = New Dictionary(Of String, String) From {
            {"Content-Type", "application/json"}}
    }
End Function
```

Line for line the same program, and a test proves it. Both take the raw `APIGatewayHttpApiV2ProxyRequest`, deserialise with `JsonSerializerOptions.Web` (camelCase, case-insensitive), parse the coverage case-insensitively and serialise the VB `PremiumBreakdown`. `HandlerParityTests` sends four bodies — including an unknown coverage and a declined quote, the 422 — to both and asserts equal status codes and byte-identical bodies. VB spells `If(a, b)` for `??`, `[Enum]` for the keyword clash and `From { }` for the dictionary initialiser; its handler string is `L12.QuoteLambdaVb::L12.QuoteLambdaVb.QuoteHandler::FunctionHandler`.

3.5 · SnapStart hooks — warm before the snapshot, renew after restore

C# — samples/L12.QuoteLambda/Startup.cs · #region snapstart

```
// SnapStart snapshots ONE initialised environment and resumes many copies of it.
public static void RegisterHooks()
{
    // Before the snapshot: drive a request through the raw handler (JSON in, rating,
    // JSON out) so restored copies skip its JIT. Call it more than once (tiered
    // compilation) and cause no side effects: this handler saves nothing.
    SnapshotRestore.RegisterBeforeSnapshot(() =>
    {
        var body = JsonSerializer.Serialize(
            new QuoteRequest("class1", 550_000m, 23, 4, 0, false, 1500),
            JsonSerializerOptions.Web);
        var request = new APIGatewayHttpApiV2ProxyRequest { Body = body };
        var handler = new RawQuoteHandler();
        for (var i = 0; i < 10; i++)
            handler.FunctionHandler(request, null!);
        return ValueTask.CompletedTask;
    });

    // After restore: every copy starts with the same memory, so renew anything unique.
    SnapshotRestore.RegisterAfterRestore(() =>
    {
        EnvironmentId = NewId();
        return ValueTask.CompletedTask;
    });
}
```

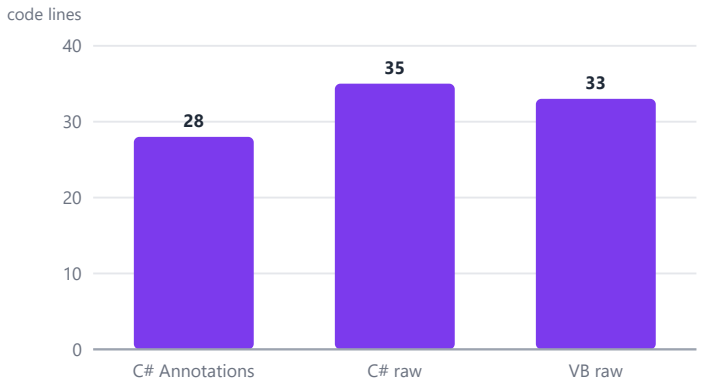
Two hooks, two different jobs. The before-snapshot hook sends a request through the raw handler ten times — JSON in, rating, JSON out — so that code is already compiled in the snapshot. AWS's .NET guidance calls the handler repeatedly because of tiered compilation, and warns that warm-up calls must have no side effects (Maximize SnapStart performance); this handler saves nothing. It does not warm the generated wrapper's DI scope or body binding, because the wrapper's constructor is what runs `Startup`: a production hook should drive the real handler where it can. The after-restore hook renews `EnvironmentId`, which would otherwise be the same GUID in every restored environment. Register hooks in the handler's constructor path — here `Startup`. The unit tests run the registration; only Lambda runs the hooks.

3.6 · .NET runtimes on Lambda

Runtime	Identifier	OS	Deprecation	Block create	Block update
.NET 10	`dotnet10`	Amazon Linux 2023	● 14 Nov 2028	14 Dec 2028	15 Jan 2029
.NET 9 — container image only	`dotnet9` · image only	Amazon Linux 2023	● 10 Nov 2026	not scheduled	not scheduled
.NET 8	`dotnet8`	Amazon Linux 2023	● 10 Nov 2026	1 Feb 2027	3 Mar 2027
.NET 6	`dotnet6`	Amazon Linux 2	● 20 Dec 2024	1 Feb 2027	3 Mar 2027
OS-only (Native AOT option)	`provided.al2023`	Amazon Linux 2023	● 30 Jun 2029	31 Jul 2029	31 Aug 2029

Four of these five identifiers are values for `runtime` =. AWS ships .NET 9 only as a container base image, so a zip function cannot name it and a container-image function sets no runtime at all. Dates re-checked against the AWS table on 2026-09-20 (Lambda runtimes).

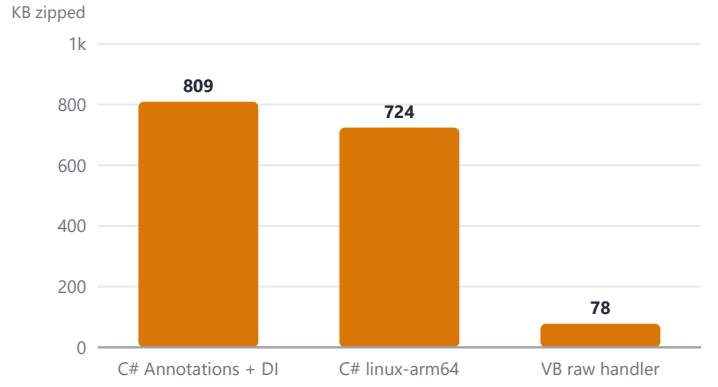
3.7 · Handler code lines — Annotations vs by hand



non-blank, non-comment lines in regions annotations, raw-cs and raw-vb · measured at build

Annotations writes about 20% fewer lines than the hand-written C# handler. The binding, the 400 for an unreadable body and the response envelope move into generated code; the raw handlers in 3.4 show the work it takes over — work VB must still write. • Measured

3.8 · Deployment package size



dotnet publish -c Release, folder zipped with deflate · captured on the build machine 2026-09-20

The DI host, not .NET, makes the package 10 times larger. The Annotations function pulls in `Microsoft.Extensions.Hosting`: 40 files and 809 KB, against 9 files and 78 KB for the raw VB handler. Publishing for `linux-arm64` drops the Windows-only EventLog assets. Set that against the 55 KB zipped Python functions you shipped. • Measured

3.9 · Testing the class Lambda actually calls

C# — samples/L12.QuoteLambda.Tests/QuoteLambdaTests.cs · #region wrapper-test

```
[Fact]
public void The_generated_handler_maps_an_http_api_event()
{
    // the class Lambda actually instantiates: written by the Annotations source generator
    var handler = new QuoteFunctions_CreateQuote_Generated();
    var request = new APIGatewayHttpApiV2ProxyRequest
    {
        Body = """{"coverage":"Class1","sumInsured":800000,"driverAge":30}""",
    };

    using var stream = handler.CreateQuote(request, new TestLambdaContext());
    var response = JsonSerializer.Deserialize<APIGatewayHttpApiV2ProxyResponse>(stream);

    Assert.Equal(201, response!.StatusCode);
    Assert.StartsWith("/quotes/", response.Headers["location"]);
}
```

Test the generated wrapper, not only your method. Constructing `QuoteFunctions_CreateQuote_Generated` runs `Startup` and the SnapStart registration exactly as INIT would; the test then feeds it an HTTP API v2 event and reads the serialised response stream. It catches what a direct call to `CreateQuote` cannot: a broken DI registration, a body-binding change, a missing `Location` header. The lesson's two test projects hold 22 test cases (lesson 10 (Testing, TDD & 100% Coverage) covers the framework).

4 · Containers without a Dockerfile

The .NET SDK builds OCI images itself: `dotnet publish -t:PublishContainer`. MSBuild properties replace the Dockerfile — `ContainerRepository`, `ContainerFamily`, `ContainerImageTags`. Listing several `ContainerRuntimeIdentifiers` produces a multi-architecture image index (SDK 8.0.405 and 9.0.102 onwards); `ContainerRegistry` pushes straight to ECR and `ContainerArchiveOutputPath` writes a tarball with no Docker daemon. There is no `RUN`: anything a Dockerfile would install belongs in a custom base image (Containerize a .NET app reference).

The image defaults changed twice, and both changes break copied Dockerfile habits. Since .NET 8, official images listen on port 8080 and ship a non-root app user (Default ASP.NET Core port changed from 80 to 8080). Since .NET 10, the default tags are Ubuntu 24.04 “Noble” and no Debian images ship (Default .NET container tags now use Ubuntu). Chiseled images remove the shell and package manager, and the size-optimised ones carry no ICU or tzdata — hence `InvariantGlobalization` in the project file (.NET container images).

Graceful shutdown is an agreement between two timeouts. ECS sends SIGTERM, waits the container's `stopTimeout` — 30 s unless set, 120 s at most — and then sends SIGKILL (ECS task definition parameters). The generic host turns SIGTERM into `ApplicationStopping`, stops accepting connections and waits `HostOptions.ShutdownTimeout` — 30 s by default — for in-

flight requests (.NET Generic Host in ASP.NET Core). Equal defaults leave no margin, so the sample sets 20 s and a test fails if anyone raises it to 30.

4.1 · The project file is the Dockerfile

MSBuild / XML — samples/L12.QuoteContainer/L12.QuoteContainer.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <!-- chiseled images carry no ICU: the app must run globalization-invariant -->
    <InvariantGlobalization>true</InvariantGlobalization>
  </PropertyGroup>

  <!-- read by: dotnet publish -c Release /t:PublishContainer  (no Dockerfile) -->
  <PropertyGroup>
    <ContainerRepository>motorquote/quote-api</ContainerRepository>
    <ContainerFamily>noble-chiseled</ContainerFamily>
    <RuntimeIdentifiers>linux-x64;linux-arm64</RuntimeIdentifiers>
    <ContainerRuntimeIdentifiers>linux-x64;linux-arm64</ContainerRuntimeIdentifiers>
    <ContainerImageTags>1.0.0;latest</ContainerImageTags>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Amazon.Lambda.AspNetCoreServer.Hosting" Version="2.2.1" />
    <PackageReference Include="OpenTelemetry.Extensions.Hosting" Version="1.18.0" />
    <PackageReference Include="OpenTelemetry.Instrumentation.AspNetCore" Version="1.18.0" />
    <PackageReference Include="OpenTelemetry.Exporter.OpenTelemetryProtocol" Version="1.18.0" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\L12.RatingVb\L12.RatingVb.vbproj" />
  </ItemGroup>
</Project>
```

Read the second property group as a Dockerfile. `ContainerFamily` `noble-chiseled` picks `aspnet:10.0-noble-chiseled` as the base; the two RID lists make one image per architecture plus an index; the tags name the image. The Lambda hosting package lets the same API run on Lambda — `AddAWSLambdaHosting` replaces Kestrel only inside a Lambda environment (Deploy ASP.NET applications to Lambda). The API calls the same VB rating library as the functions.

4.2 · Two hosting targets, one shutdown budget

C# — samples/L12.QuoteContainer/Program.cs · #region hosting

```
// Kestrel in the container. The Lambda adapter replaces Kestrel only when the
// process runs inside Lambda: one codebase, two hosting targets.
builder.Services.AddAWSLambdaHosting(LambdaEventSource.HttpApi);

// ECS sends SIGTERM, waits stopTimeout (30 s by default), then SIGKILL.
// Finish in-flight requests well inside that window.
builder.Services.Configure<HostOptions>(
    o => o.ShutdownTimeout = TimeSpan.FromSeconds(20));
```

One codebase, two hosts, one rule. On ECS the process is Kestrel on port 8080; on Lambda the adapter swaps the server and the HTTP API event becomes an `HttpContext`. The 20-second `ShutdownTimeout` is checked by `Shutdown_timeout_fits_inside_the_ecs_stop_timeout`, so the ECS `stopTimeout` of 30 s in `main.tf` always has headroom.

4.3 · Liveness and readiness without a shell

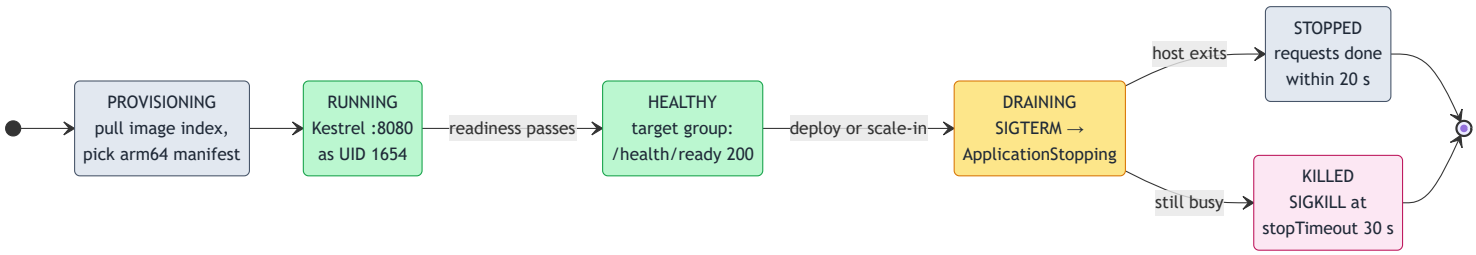
C# — samples/L12.QuoteContainer/Program.cs · #region health

```
// liveness = the process answers; readiness = its dependencies work
app.MapHealthChecks("/health/live", new HealthCheckOptions
    { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions
    { Predicate = check => check.Tags.Contains("ready") });

app.Lifetime.ApplicationStopping.Register(() =>
    app.Logger.LogInformation("Stop signal received: draining requests"));
```

Two endpoints, two questions. `/health/live` runs no checks — the process answers, so do not restart it. `/health/ready` runs the checks tagged `ready`; here `RatingSelfCheck` prices a quote through the VB library. ECS container health checks run a command inside the container (`CMD` or `CMD-SHELL`), and a chiseled image has neither a shell nor curl — so the load balancer's target group probes `/health/ready` from outside (5.5).

4.4 · An ECS task's life, from pull to SIGKILL



green = serving · amber = draining · rose = forced stop · the numbers are this lesson's settings: HostOptions 20 s inside an ECS stopTimeout of 30 s

4.5 · What you should see — two images and an index

Terminal — image, no Dockerfile, no daemon — captured 2026-09-20 · edited: paths, wrapping, one publish line, MSBuild prefixes

```
$ dotnet publish $SAMPLES/L12.QuoteContainer -c Release -t:PublishContainer \
    -p ContainerArchiveOutputPath=./images/
  L12.QuoteContainer -> .build/artifacts/publish/L12.QuoteContainer/release_linux-arm64/
  L12.QuoteContainer -> .build/artifacts/publish/L12.QuoteContainer/release_linux-x64/
  Building image 'motorquote/quote-api' for runtime identifier 'linux-x64'
    on top of base image 'mcr.microsoft.com/dotnet/aspnet:10.0-noble-chiseled'.
  Building image 'motorquote/quote-api' for runtime identifier 'linux-arm64'
    on top of base image 'mcr.microsoft.com/dotnet/aspnet:10.0-noble-chiseled'.
  Building image index 'motorquote/quote-api:1.0.0, latest' on top of manifests
    sha256:477d0822727a…, sha256:e2539bf83493….
  Pushed image 'motorquote/quote-api:1.0.0, latest' to local archive at
    './images/motorquote/quote-api.tar.gz'.
```

No Docker, no Dockerfile. The SDK publishes the app once per RID, downloads the chiseled base image from mcr.microsoft.com, adds one layer with your files and writes both images plus an OCI image index into the archive. Swap `ContainerArchiveOutputPath` for `ContainerRegistry` and the same command pushes to ECR, as the pipeline in 5.3 does. In Git Bash on Windows write `-t:`: the shell rewrites `/t:PublishContainer` as a file path and MSBuild stops with “Only one project can be specified”.

4.6 · What is inside the image, per architecture



compressed layer sizes in MB, read from the image manifests in the archive · captured 2026-09-20 · the MotorQuote app layer is 0.55 MB — 1% of the image, a sliver at this scale

Your code is 1.0% of the image — a 0.55 MB layer on 55.1 MB of base. The base layers are shared by every .NET 10 API in the registry and on the host, so a deploy moves only the top layer. The arm64 image is 52.0 MB against 55.7 MB for x64; the two tiny layers (the `/home/app` folder and the `dotnet` symlink) are folded into their neighbours. • Measured

4.7 · The image configuration the SDK wrote

Setting	Value in the linux-x64 manifest	Where it came from
Base image	`mcr.microsoft.com/dotnet/aspnet:10.0-noble-chiseled`	`ContainerFamily` + the `net10.0` TFM
User	`1654` (`APP_UID`)	the base image's non-root app user
Port	`ASPNETCORE_HTTP_PORTS=8080` · exposed `8080/tcp`	base image variable; the SDK infers the exposed port from it
Globalization	`DOTNET_SYSTEM_GLOBALIZATION_INVARIANT=true`	chiseled base without ICU — matches `InvariantGlobalization`
Entry point	`dotnet /app/L12.QuoteContainer.dll` · working dir `/app/`	the SDK, framework-dependent publish
Runtime	`DOTNET_VERSION=10.0.12` · `ASPNET_VERSION=10.0.12`	the base image on the day of the build
Tags	`1.0.0` · `latest` → one OCI index over amd64 + arm64	`ContainerImageTags` + `ContainerRuntimeIdentifiers`

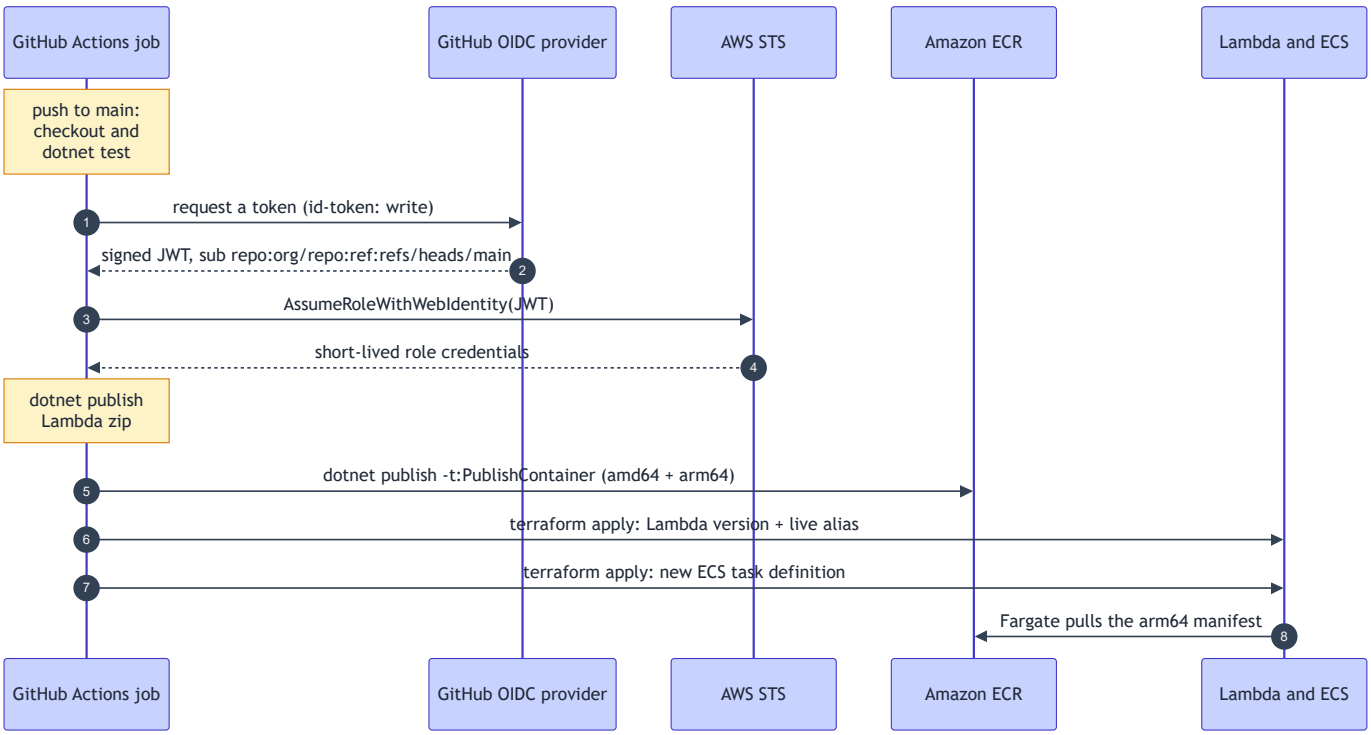
5 · Infrastructure and delivery — Terraform, OIDC, CDK and Aspire

Your Terraform and GitHub Actions carry over; three .NET details do not. The runtime is `dotnet10` and, with Annotations, the handler names the generated class. SnapStart needs `publish = true` and an alias, because it applies only to published versions. And an AWS provider build from before the runtime existed rejects `dotnet10` at validate time — exactly what users reported in January 2026 (terraform-provider-aws #45888). The sample's `~> 6.0` floats: on 2026-09-16 it resolved to hashicorp/aws 6.64.0, whose runtime list includes `dotnet10`, and a one-off `terraform validate` on that copy passed — run by hand, not repeated by the build, so read it as a dated observation, not a guarantee.

The pipeline authenticates with OIDC and stores no AWS keys. `id-token: write` lets GitHub mint a token that `configure-aws-credentials` exchanges for a role session; the role's trust policy pins the audience `sts.amazonaws.com` and a `sub` such as `repo:org/repo:ref:refs/heads/main` (Configuring OpenID Connect in AWS). Identity and secrets in depth: lesson 11 (Security & Identity with Entra ID). Action majors in the sample were the latest releases on 2026-09-16: actions/checkout@v7 · actions/setup-dotnet@v6 · aws-actions/configure-aws-credentials@v6 · aws-actions/amazon-ecr-login@v2 · hashicorp/setup-terraform@v4.

C# teams have two more infrastructure tools. The AWS CDK treats .NET as a stable, fully supported language (.NET 8 or later; the CDK CLI still needs Node.js) (Working with the AWS CDK in C#). Aspire — renamed from ".NET Aspire" at version 13 on 11 November 2025, and needing the .NET 10 SDK (What's new in Aspire 13) — orchestrates the local loop; its AWS integration provisions resources through CloudFormation and, in preview, runs Lambda functions locally (Integrating AWS with Aspire).

5.1 · From push to running, with no stored keys



the only credentials are the ones STS returns in step 4, and they expire · the token's sub claim is what the role's trust policy matches

5.2 · The workflow — test, authenticate, package

YAML `samples/ci/deploy.yml · #region build`

```yaml
- uses: actions/checkout@v7
- uses: actions/setup-dotnet@v6
  with:
    global-json-file: global.json

- run: dotnet test $SAMPLES/L12.QuoteLambda.Tests -c Release

- uses: aws-actions/configure-aws-credentials@v6
  with:
    role-to-assume: ${{ vars.DEPLOY_ROLE_ARN }} # no access keys stored anywhere
    aws-region: ap-southeast-1

- name: Package the Lambda (framework-dependent IL runs on arm64 unchanged)
  run: |
    dotnet publish $SAMPLES/L12.QuoteLambda -c Release -o out/lambda
    (cd out/lambda && zip -qr ../quote-lambda.zip .)
```

> **The SDK version comes from the repository, the credentials from a role.** `setup-dotnet` reads `global.json`, so CI builds with the SDK your laptop pins (lesson 07 (Solutions, MSBuild & Mono-repos)). `role-to-assume` is the only AWS setting — no access key exists to leak. The Lambda zip is framework-dependent IL, so it runs on the `arm64` function although the runner is x64.

5.3 · The workflow — push the image, apply the plan

YAML — samples/ci/deploy.yml · #region ship

```yaml
- id: ecr
  uses: aws-actions/amazon-ecr-login@v2

- name: Build and push a multi-arch image (no Dockerfile, no Docker daemon)
  run: >
    dotnet publish $SAMPLES/L12.QuoteContainer -c Release -t:PublishContainer
    -p ContainerRegistry=${{ steps.ecr.outputs.registry }}
    -p ContainerImageTags=${{ github.sha }}

- uses: hashicorp/setup-terraform@v4
- run: terraform -chdir=$SAMPLES/infra init -input=false
- run: >
    terraform -chdir=$SAMPLES/infra apply -input=false -auto-approve
    -var lambda_zip=${{ github.workspace }}/out/quote-lambda.zip
    -var image=${{ steps.ecr.outputs.registry }}/motorquote/quote-api:${{ github.sha }}
```

No Docker build step, no Dockerfile in the repository. `amazon-ecr-login` gives the SDK registry credentials; `ContainerRegistry` redirects the publish from 4.5 to ECR, and the commit SHA replaces the tags. Terraform receives the zip path and the image reference as variables, so one apply moves both targets. The file sits under `samples/ci/`, where GitHub never runs it.

5.4 · Terraform — the function, SnapStart and a live alias

Terraform — samples/infra/main.tf · #region lambda

```terraform
resource "aws_lambda_function" "create_quote" {
  function_name    = "motorquote-create-quote"
  role             = var.lambda_role_arn
  runtime          = "dotnet10"
  architectures    = ["arm64"]
  memory_size      = 1024
  timeout          = 10
  filename         = var.lambda_zip
  source_code_hash = filebase64sha256(var.lambda_zip)
  handler          = "${local.asm}::${local.asm}.QuoteFunctions_CreateQuote_Generated::CreateQuote"

  publish = true # SnapStart snapshots published versions, never $LATEST
  snap_start {
    apply_on = "PublishedVersions"
  }

  logging_config {
    log_format = "JSON" # message-template arguments become JSON fields
  }
}

resource "aws_lambda_alias" "live" {
  name             = "live"
  function_name    = aws_lambda_function.create_quote.function_name
  function_version = aws_lambda_function.create_quote.version
}
```

Three lines carry the .NET decisions. `runtime = "dotnet10"` with `architectures = ["arm64"]` runs the same IL on Graviton. The handler string names the generated wrapper tested in 3.9 — name `QuoteFunctions` instead and the function fails when it initialises, not when Terraform applies. `publish = true` plus `snap_start` snapshots each published version; the HTTP API integrates with the `live` alias (in `main.tf`, not printed), never `$LATEST`. `log_format = "JSON"` turns message-template arguments into fields.

5.5 · Terraform — the Fargate task and its target group

Terraform samples/infra/main.tf · #region ecs

```
resource "aws_ecs_task_definition" "api" {
  family                   = "motorquote-api"
  requires_compatibilities = ["FARGATE"]
  network_mode             = "awsvpc"
  cpu                      = 512
  memory                   = 1024
  execution_role_arn       = var.task_execution_role_arn

  runtime_platform {
    operating_system_family = "LINUX"
    cpu_architecture        = "ARM64"
  }

  container_definitions = jsonencode([{
    name         = "quote-api"
    image        = var.image
    essential    = true
    portMappings = [{ containerPort = 8080 }] # .NET 8+ images listen on 8080
    stopTimeout  = 30                         # SIGTERM, wait, then SIGKILL
    # no container healthCheck: a chiseled image has no shell and no curl
  }])
}

resource "aws_lb_target_group" "api" {
  name        = "motorquote-api"
  port        = 8080
  protocol    = "HTTP"
  target_type = "ip"
  vpc_id      = var.vpc_id

  health_check {
    path = "/health/ready" # the load balancer probes readiness from outside
  }
}
```

The container side repeats §4 in HCL. `cpu_architecture = "ARM64"` picks the arm64 manifest from the image index, `containerPort = 8080` matches the image, `stopTimeout = 30` is the budget the host's 20 s fits in, and the target group probes `/health/ready` because the image has no shell. The VPC, cluster, service, listener and IAM roles arrive as variables; the collector sidecar and the `OTEL_EXPORTER_OTLP_ENDPOINT` that points the API at it are left out too, so as written this task exports no telemetry (6.1 shows the app side). A sketch that validates, not a stack that was applied.

5.6 · Three ways to describe the same infrastructure

Terraform (HCL)

- What the sample uses — your existing estate, state and modules carry over
- Runtime strings validated by the provider: pin a build that knows `dotnet10`
- Language-neutral: the .NET build stays in `dotnet publish`

AWS CDK in C#

- Infrastructure as C# classes and props objects — `Amazon.CDK.Lib`
- Stable and fully supported for .NET; VB and F# work with limited support
- Synthesises CloudFormation; the CLI still needs Node.js

Aspire

- Local orchestration and a telemetry dashboard for multi-service apps
- Aspire 13 dropped the ".NET" prefix and requires the .NET 10 SDK
- `Aspire.Hosting.AWS`: CloudFormation provisioning; local Lambda runs in preview

6 · Observability — OpenTelemetry on .NET's own APIs

In .NET the OpenTelemetry API is already in the base library. `ActivitySource` and `Activity` are the tracer and the span, `System.Diagnostics.Metrics.Meter` is the meter, and `ILogger` is the logger; the OpenTelemetry packages subscribe to them by name and export (.NET observability with OpenTelemetry). A library can emit spans without referencing OpenTelemetry at all — the reverse of adding the Java agent or Micrometer to a Spring service.

On AWS, export OTLP to a collector and let it reach X-Ray and CloudWatch. The X-Ray SDKs and daemon entered maintenance mode on 25 February 2026; AWS recommends OpenTelemetry instrumentation with an OpenTelemetry collector (such as ADOT's) or the CloudWatch agent in place of the daemon, and converts server spans to segments and attributes to

metadata (Migrating from X-Ray instrumentation to OpenTelemetry). The sample exports only when `OTEL_EXPORTER_OTLP_ENDPOINT` is set, so tests and laptops need no collector.

On Lambda, structured logs are a setting on the function, not a package. With the log format set to JSON, Lambda turns the `{QuoteId}` placeholders passed to `context.Logger` into top-level JSON fields in CloudWatch Logs (Log and monitor C# Lambda functions); why the message must be a template and not an interpolated string is the `ILogger` rule of lesson 08 (ASP.NET Core Web APIs). Powertools for AWS Lambda (.NET) adds cold-start fields, EMF metrics and idempotency on top (Powertools for AWS Lambda (.NET)).

6.1 · Wiring traces, metrics and logs

C# — samples/L12.QuoteContainer/Program.cs · #region telemetry

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("motorquote-api"))
    .WithTracing(t => t
        .AddSource(QuoteTelemetry.Source.Name)
        .AddAspNetCoreInstrumentation())
    .WithMetrics(m => m
        .AddMeter(QuoteTelemetry.Meter.Name)
        .AddAspNetCoreInstrumentation())
    .WithLogging();

// Export over OTLP only when a collector is configured, e.g. an ADOT or
// CloudWatch agent sidecar on ECS listening on http://localhost:4317.
if (builder.Configuration["OTEL_EXPORTER_OTLP_ENDPOINT"] is not null)
    builder.Services.AddOpenTelemetry().UseOtlpExporter();
```

Subscribe by name, export by configuration. `AddSource` and `AddMeter` name the `ActivitySource` and `Meter` declared in `QuoteTelemetry`; the ASP.NET Core instrumentation adds a server span and HTTP metrics per request. `UseOtlpExporter` sends all three signals to one OTLP endpoint — on ECS, a CloudWatch agent or ADOT collector sidecar on `localhost:4317`. The resource name `motorquote-api` is the `service.name` every backend groups by.

6.2 · One request: a span, a counter and a structured log

C# — samples/L12.QuoteContainer/Program.cs · #region endpoint

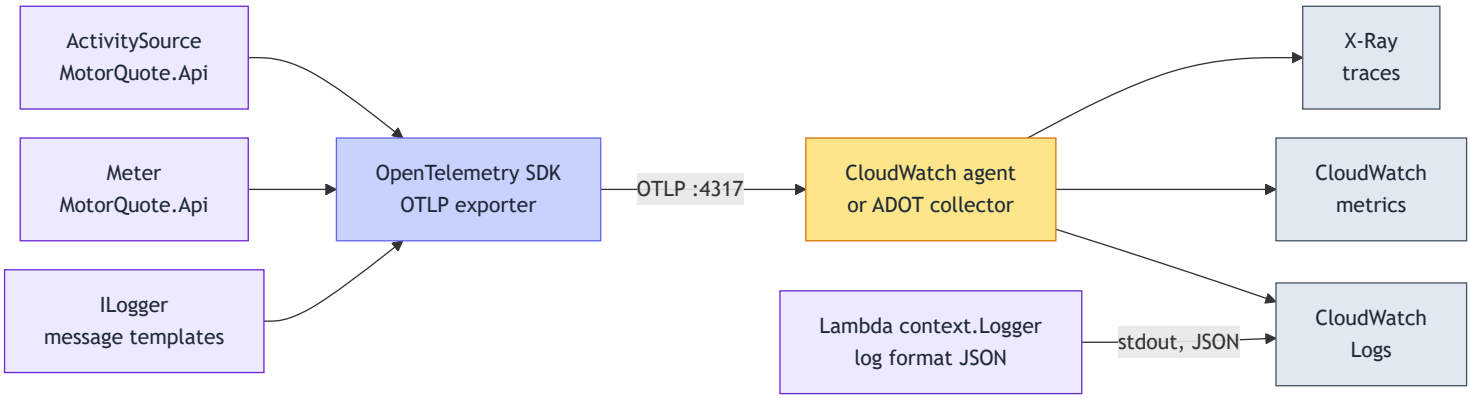
```
app.MapPost("/quotes", IResult (QuoteRequest req, ILogger<QuoteRequest> log) =>
{
    using var span = QuoteTelemetry.Source.StartActivity("rate-quote");
    span?.SetTag("motorquote.coverage", req.Coverage.ToString());

    PremiumBreakdown p;
    try
    {
        p = Rating.Quote(req.Coverage, req.SumInsured, req.DriverAge,
                         req.LicenceYears, req.ClaimsLast5Years,
                         req.Commercial, req.EngineCc);
    }
    catch (QuoteDeclinedException declined)
    {
        return TypedResults.UnprocessableEntity(declined.Message);
    }

    QuoteTelemetry.QuotesIssued.Add(1,
        new KeyValuePair<string, object?>("coverage", req.Coverage.ToString()));
    // a message TEMPLATE, not an interpolated string: Coverage and Total
    // become searchable fields in CloudWatch Logs
    log.LogInformation("Quoted {Coverage}: total {Total} THB", req.Coverage, p.Total);
    return TypedResults.Ok(p);
});
```

Three signals, three APIs, no vendor type. `StartActivity` returns `null` when nothing listens — hence `span?.`. The counter's `coverage` tag becomes a metric dimension, so keep its values few. The log call passes `Coverage` and `Total` as template arguments, so a backend can filter on them — the template rule is in lesson 08 (ASP.NET Core Web APIs). The premium comes from the VB rules, as in every other host in this lesson; a declined quote (three or more claims) returns 422 before the counter and the log line run.

6.3 · Where each signal goes



violet = .NET APIs in your code · indigo = OpenTelemetry SDK · amber = collector · slate = AWS backends · the Lambda path needs no collector for logs

6.4 · Asserting on spans in a test

C# — samples/L12.QuoteContainer.Tests/ContainerApiTests.cs · #region span-test

```
[Fact]
public async Task A_quote_emits_a_server_span_and_a_domain_span()
{
    var spans = new List<Activity>();
    using var app = factory.WithWebHostBuilder(web => web.ConfigureTestServices(
        services => services.ConfigureOpenTelemetryTracerProvider(
            tracing => tracing.AddInMemoryExporter(spans))));

    var response = await app.CreateClient().PostAsJsonAsync("/quotes", Request);
    response.EnsureSuccessStatusCode();

    // the server span stops after the response is written: wait for it
    SpinWait.SpinUntil(() => spans.ToArray().Any(s => s.Kind == ActivityKind.Server),
                       TimeSpan.FromSeconds(5));
    Assert.Contains(spans.ToArray(), s => s.DisplayName == "rate-quote");
    Assert.Contains(spans.ToArray(), s => s.Kind == ActivityKind.Server);
}
```

Telemetry is behaviour, so it gets a test. `ConfigureTestServices` adds an in-memory exporter to the tracer provider the app already configured, and the test asserts that both the domain span `rate-quote` and the ASP.NET Core server span were exported. The server span ends after the response is written, so the test waits for it — asserting straight away would race the server span.

7 · Modernising Windows and IIS estates

Triage the estate with the AWS 7 Rs — not six. AWS Prescriptive Guidance lists Retire, Retain, Rehost, Relocate, Repurchase, Replatform and Refactor, and for large migrations recommends rehosting, relocating or replatforming first and modernising afterwards, because refactoring during the move is the most complex path (About the migration strategies). For a .NET estate the deciding signals sit in each repository: the target framework, `System.Web`, WCF, COM and the GAC, MSMQ, Web Forms. `L12.EstateTriage` turns one signal into one rule.

Port by API family, not by project. Application domains, remoting, code access security and COM+ do not exist on .NET 10; WCF services move to CoreWCF for existing SOAP clients or to gRPC for new ones (.NET Framework technologies unavailable on .NET 6+ · CoreWCF 1.0 has been released). The route for a large ASP.NET application is incremental: an ASP.NET Core app in front proxies every route not yet migrated to the Framework app with YARP, and the System.Web adapters let shared libraries keep using `HttpContext` while routes move (Migrate from ASP.NET Framework to ASP.NET Core · Get started with incremental migration). Web Forms has no port — its pages are rebuilt, typically in Blazor (Migrate from ASP.NET Web Forms to Blazor). VB6 is COM, not .NET: lesson 06 (VB.NET for Legacy Estates).

Tools now plan and rewrite code; they do not make the decisions. .NET Upgrade Assistant is deprecated in favour of GitHub Copilot app modernization (Upgrade .NET apps overview). AWS Transform for .NET moves .NET Framework 3.5+ code to .NET 8 or .NET 10 on Linux; the project types it lists include MVC, Web API, Web Forms and WCF services, with VB.NET in preview (Modernizing .NET with AWS Transform). Both produce a plan and a branch; the 7 R call and the target stay with you.

7.1 · Porting map — .NET Framework on IIS → .NET 10

In the Framework app	On .NET 10	Kind	Watch for
ASP.NET MVC 5 · Web API 2 (`System.Web`)	ASP.NET Core controllers or minimal APIs — lesson 08 (ASP.NET Core Web APIs)	• Different	Migrate route by route; System.Web adapters for shared `HttpContext` code
Web Forms (`.aspx`)	Blazor or Razor Pages	• Trap	No port: pages are rebuilt, events become components
WCF service (`.svc`)	CoreWCF · gRPC for new callers	• Renamed	1.0 shipped HTTP and NetTCP bindings: check your bindings against the current release
`web.config` app settings · `Global.asax`	`appsettings.json` + environment · `Program.cs` middleware	• Renamed	Secrets move to a secrets manager — lesson 11 (Security & Identity with Entra ID)
HTTP modules and handlers	Middleware	• Renamed	Order matters and is explicit in code
Windows service · MSMQ	`BackgroundService` worker · Amazon SQS	• Different	MSMQ does not exist on Linux: the queue is an architecture change
Application domains · .NET Remoting	Processes or containers · gRPC, HTTP, StreamJsonRpc	• Different	`AppDomain.CreateDomain` throws `PlatformNotSupportedException`
Code access security · COM+ (`EnterpriseServices`)	OS, container and IAM boundaries	• Trap	No replacement API — redesign the boundary
Windows Workflow Foundation	CoreWF (community port) or Step Functions	• Different	Not supported on .NET 6+ — decide per workflow

7.2 · The triage rules — one signal, one arm

C# — samples/L12.EstateTriage/Triage.cs · #region rules

```
// First matching arm wins — order the arms from "stop" to "default".
public static Decision Decide(App app) => app switch
{
    { Value: "none" } =>
        new(Strategy.Retire, "-", "archive the data, switch it off"),
    { Kind: "cots" } =>
        new(Strategy.Repurchase, "SaaS", "replace with the vendor's SaaS"),
    { Framework: "vb6", Kind: "desktop" } =>
        new(Strategy.Refactor, "desktop", "rewrite in C# for .NET: VB6 is COM, not .NET"),
    { Framework: "vb6" } =>
        new(Strategy.Refactor, "ECS Fargate", "rewrite in C# as a service: VB6 is COM"),
    { Signals: var s } when s.Contains("com-interop") || s.Contains("gac") =>
        new(Strategy.Rehost, "EC2 Windows/IIS", "lift as-is; port once COM/GAC is gone"),
    { Signals: var s } when s.Contains("msmq") =>
        new(Strategy.Replatform, "ECS Fargate", "BackgroundService; MSMQ becomes SQS"),
    { Kind: "winforms" } =>
        new(Strategy.Retain, "desktop", "retarget net10.0-windows in place"),
    { Framework: "net8.0" or "net9.0" } =>
        new(Strategy.Replatform, "ECS Fargate", "retarget net10.0 + container publish"),
    { Kind: "wcf" } =>
        new(Strategy.Replatform, "ECS Fargate", "CoreWCF; gRPC for new callers"),
    { Kind: "webforms" } =>
        new(Strategy.Refactor, "ECS Fargate", "Blazor pages strangled behind YARP"),
    { Kind: "batch", Traffic: "spiky" } =>
        new(Strategy.Refactor, "Lambda", "event-driven function per job"),
    { Kind: "mvc" or "webapi" } =>
        new(Strategy.Replatform, "ECS Fargate", "ASP.NET Core + System.Web adapters"),
    _ =>
        new(Strategy.Replatform, "ECS Fargate", "port to net10.0"),
};
```

> **The order of the arms is the policy.** A C# switch expression takes the first matching arm, so "stop" decisions (retire, buy, rewrite VB6) come before "can we lift it" (COM or GAC → rehost on Windows) and before the ports. Property patterns read like the inventory columns; `when` guards test the signal set. The inventory is illustrative, and the rules are a starting point to argue with, not an assessment method.

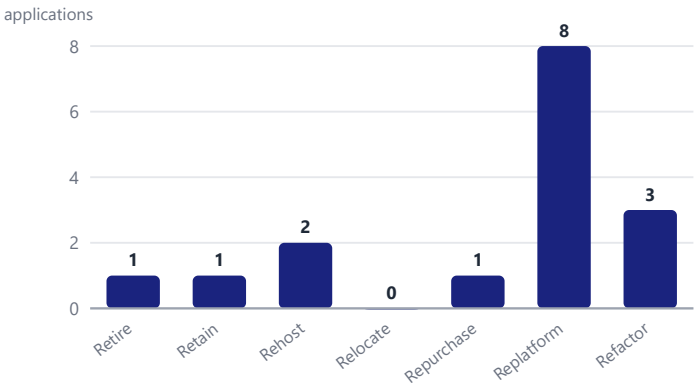
7.3 · What you should see — sixteen applications, seven Rs

Output — L12.EstateTriage captured on the build machine 2026-09-20

```
Triage of 16 applications (illustrative inventory)
  policy-portal     net48   Refactor   ECS Fargate      Blazor pages strangled behind YARP
  quote-engine      net472  Replatform ECS Fargate      ASP.NET Core + System.Web adapters
  partner-gateway   net48   Replatform ECS Fargate      CoreWCF; gRPC for new callers
  claims-intake     net48   Replatform ECS Fargate      ASP.NET Core + System.Web adapters
  broker-desktop    net48   Retain     desktop          retarget net10.0-windows in place
  premium-batch     net48   Refactor   Lambda           event-driven function per job
  renewal-notices   net472  Replatform ECS Fargate      BackgroundService; MSMQ becomes SQS
  doc-generator     net40   Rehost     EC2 Windows/IIS  lift as-is; port once COM/GAC is gone
  agent-commission  vb6     Refactor   desktop          rewrite in C# for .NET: VB6 is COM, not .NET
  rates-admin       net8.0  Replatform ECS Fargate      retarget net10.0 + container publish
  payment-callbacks net8.0  Replatform ECS Fargate      retarget net10.0 + container publish
  legacy-reports    net35   Retire     -                archive the data, switch it off
  hr-leave          -       Repurchase SaaS             replace with the vendor's SaaS
  fx-rates-feed     net472  Replatform ECS Fargate      port to net10.0
  sms-dispatch      net48   Replatform ECS Fargate      port to net10.0
  vehicle-lookup    net48   Rehost     EC2 Windows/IIS  lift as-is; port once COM/GAC is gone
By strategy (7 Rs):
  Retire      1
  Retain      1
  Rehost      2
  Relocate    0
  Repurchase  1
  Replatform  8
  Refactor    3
By target:
  ECS Fargate      9
  desktop          2
  EC2 Windows/IIS  2
  Lambda           1
  -                1
  SaaS             1
```

8 of 16 applications replatform, and only 2 need Windows servers at all. The two rehosts are held back by COM and the GAC, not by C# or VB, and they can land on EC2 with IIS or in an ECS Windows container; the one refactor to Lambda is a spiky batch job; the WinForms desktop is retained and retargeted in place, and the VB6 desktop app is rewritten as a .NET desktop app (lesson 06 (VB.NET for Legacy Estates)). Relocate stays at zero: it moves servers to a cloud version of the same platform, and no row here is that.

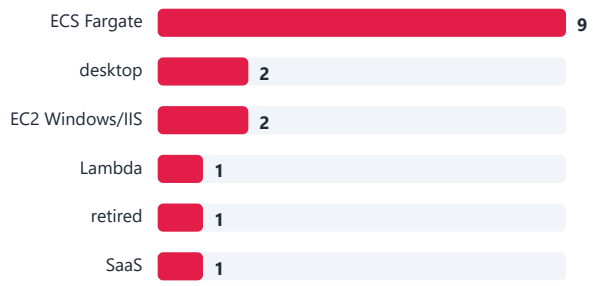
7.4 · Applications per migration strategy



parsed from the captured triage output (7.3) · illustrative inventory

Replatform dominates (8 of 16), which matches AWS's advice for large migrations. Refactor is kept for a VB6 rewrite, a Web Forms rebuild and an event-driven batch job. • Measured

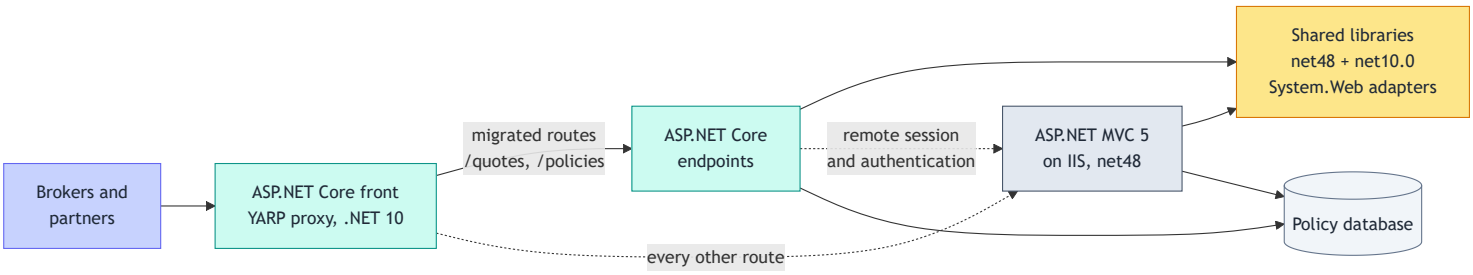
7.5 · Applications per target platform



parsed from the captured triage output (7.3) · retired = no target

ECS Fargate takes 9 of 16. Once `System.Web` is gone a Linux container is the default; Lambda gets the one workload whose traffic shape fits it. • Measured

7.6 · Strangling one IIS application behind YARP



slate = still .NET Framework on IIS · teal = ASP.NET Core on .NET 10 · amber = the shared piece · each release moves routes from the dotted path to the solid one

7.7 · Modernisation tools as of September 2026

Tool	What it does	Status
GitHub Copilot app modernization	Assesses a solution, writes an upgrade plan, applies and validates the changes; its cloud steps target Azure	● **Recommended by Microsoft** needs GitHub Copilot access
.NET Upgrade Assistant	Visual Studio extension and CLI that applies common upgrade changes	● **Deprecated** use only without Copilot access
AWS Transform for .NET	.NET Framework 3.5+ → .NET 8 or .NET 10 on Linux; MVC, Web API, Web Forms projects, WCF services; commits to a branch	● **Available** VB.NET in preview
System.Web adapters · YARP	Run Framework and Core side by side; share session and authentication	● **Current · 2.3.0** the incremental-migration toolkit
CoreWCF	Service-side WCF on .NET: SOAP over HTTP and NetTCP, WSDL	● **Current · 1.9.1** Microsoft support; gRPC for greenfield

8 · Hands-on — and what to learn after the track

7 projects, one command to prove them. `verify_samples.py` builds the VB rating library and both Lambda projects, runs 22 test cases in two test projects, builds the container API and runs the triage console to completion. None of it touches AWS.

Two commands go further than the verifier. `dotnet publish -t:PublishContainer` writes the image archive of 4.5 (it downloads the chiseled base image), and a plain `dotnet publish` of each Lambda project gives the folders whose zips 3.8 measures. Deploying for real needs an AWS account, a deploy role trusted for your repository and the variables `main.tf` declares.

After the track, go deeper where your next project is. For serverless, the AWS Lambda .NET developer guide and Powertools for AWS Lambda (.NET); for containers, the .NET container publishing reference; for a Framework estate, Microsoft's ASP.NET Framework → ASP.NET Core migration guide and AWS Transform for .NET; for the local loop and telemetry, Aspire. Keep re-verifying: Lambda runtime and SDK dates move every November.

8.1 · Commands

Terminal

```
# build, test and run every lesson-12 sample (from the repository root)
python 0-script/verify_samples.py --only 12

# the tests on their own
dotnet test lesson-12-cloud-native-modernization/samples/L12.QuoteLambda.Tests
dotnet test lesson-12-cloud-native-modernization/samples/L12.QuoteContainer.Tests

# triage the illustrative estate
dotnet run --project lesson-12-cloud-native-modernization/samples/L12.EstateTriage

# a multi-arch image archive, no Docker (Git Bash: -t:, not /t:)
dotnet publish lesson-12-cloud-native-modernization/samples/L12.QuoteContainer \
    -c Release -t:PublishContainer -p ContainerArchiveOutputPath=./images/

# the Lambda package folders measured in 3.8
dotnet publish lesson-12-cloud-native-modernization/samples/L12.QuoteLambda -c Release -o out/cs
dotnet publish lesson-12-cloud-native-modernization/samples/L12.QuoteLambdaVb -c Release -o out/vb
```

Every command runs offline except the image publish. The SDK comes from `global.json` (10.0.401 or a later 10.0 feature band). `out/` and `images/` are scratch output — do not commit them.

8.2 · What you should see

Output — verify_samples, dotnet test captured 2026-09-20 · $SAMPLES = lesson-12-…/samples · SDK path, TFM, test durations, padding cut

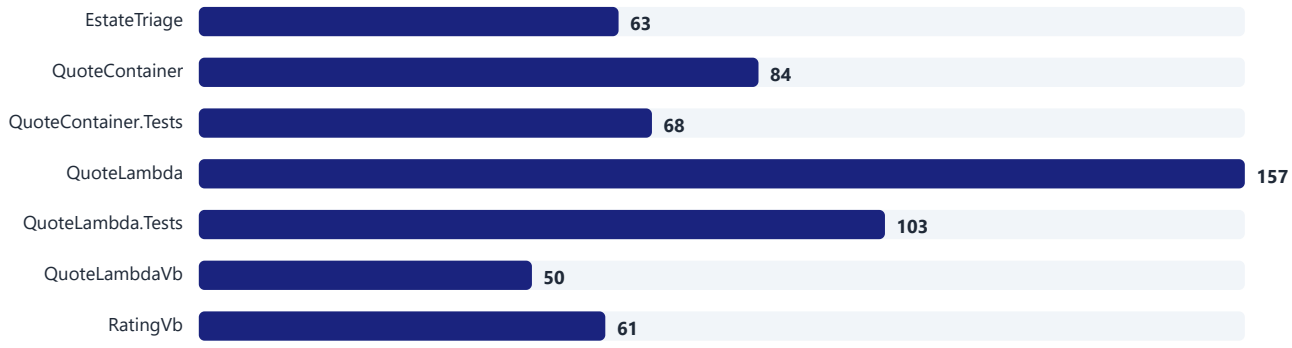
```
dotnet SDK 10.0.401
PASS  console     1.7s  $SAMPLES/L12.EstateTriage/L12.EstateTriage.csproj
PASS  web         1.4s  $SAMPLES/L12.QuoteContainer/L12.QuoteContainer.csproj
PASS  test        3.4s  $SAMPLES/L12.QuoteContainer.Tests/L12.QuoteContainer.Tests.csproj
PASS  library     1.2s  $SAMPLES/L12.QuoteLambda/L12.QuoteLambda.csproj
PASS  test        2.9s  $SAMPLES/L12.QuoteLambda.Tests/L12.QuoteLambda.Tests.csproj
PASS  library     1.1s  $SAMPLES/L12.QuoteLambdaVb/L12.QuoteLambdaVb.vbproj
PASS  library     1.0s  $SAMPLES/L12.RatingVb/L12.RatingVb.vbproj

7/7 sample projects passed

Passed!  - Failed: 0, Passed: 16, Skipped: 0, Total: 16 - L12.QuoteLambda.Tests.dll
Passed!  - Failed: 0, Passed:  6, Skipped: 0, Total:  6 - L12.QuoteContainer.Tests.dll
```

Timings will differ; the counts should not. Sixteen Lambda test cases cover five tariff totals (the first is the track's worked example, 8,933.71 THB) and the three-claims decline, the Annotations function's 201, 400, 422 and 404 answers, the generated wrapper and C# ↔ VB handler parity on four bodies; six container tests cover both health endpoints, the shutdown budget, the same VB-computed 8,933.71 THB total, the 422 decline and the exported spans. If a count differs, a sample changed — the PDF build checks the source against these numbers.

8.3 · Code lines per lesson-12 project



non-blank, non-comment C# and VB lines per project · measured at build time

The VB rating rules are 61 lines, and three hosts and 22 test cases depend on them. The largest project is `QuoteLambda` at 157 lines; the three hosts and their two test projects add up to 462 lines around that small core — the shape a modernised estate should have: move the rules once, re-host them many times. • **Measured**

⚠ Traps

1. **"Module-scope state is fine in Lambda."** It lives per execution environment, many run in parallel and any can be recycled — and on Lambda Managed Instances one handler object serves concurrent requests. Use DynamoDB for state and thread-safe types for caches.
2. **"SnapStart is free and transparent."** For .NET it bills caching and every restore; a GUID, random seed or cached credential created during INIT is identical in every restored copy; and it does not combine with provisioned concurrency or OS-only runtimes.
3. **"I can build the Native AOT package on my laptop."** The native binary must be compiled on Amazon Linux 2023 for the function's architecture — the Lambda CLI uses a build container for it — and trimmed code needs a test on Lambda, not only a passing unit test (Compile .NET Lambda code to a native runtime format).
4. **"The Dockerfile habits still apply."** .NET 8+ images listen on 8080 as UID 1654, chiseled images have no shell for `CMD-SHELL` health checks or ICU for culture data, and `HostOptions.ShutdownTimeout` equal to the ECS `stopTimeout` leaves no margin.
5. **"Terraform will accept any runtime AWS has launched."** The provider validates the runtime list it was built with; pin a version that knows `dotnet10`. And in Git Bash, `/t:PublishContainer` becomes a path — write `-t:`.

✓ Checkpoint — you have finished the track when you can

1. Run `python 0-script/verify_samples.py --only 12` and see 7/7 passed.
2. Write the handler string for a class-library function, with and without Annotations, and say what runs during INIT.
3. Pick a target for a workload from the flow in 2.2 and defend it against two other columns of 2.3.
4. Publish the API as an image archive and explain every row of table 4.7.
5. Take one row of the triage output in 7.3 and list its ports from table 7.1 and its route to production from 7.6.

🔍 Check yourself

1. What is the handler string of the Annotations function in `main.tf`, and why does it not name `QuoteFunctions`?
2. Name three things SnapStart requires or rules out, and one thing it charges .NET functions for that Java functions do not pay.
3. Why does the Terraform sample set `publish = true` and route the API to an alias?
4. Which port and user does the published image run as, and which two .NET releases changed the image defaults?
5. ECS sends SIGTERM to the API. Which host event fires, which timeout applies, and why is it 20 s in the sample?
6. Which .NET types are OpenTelemetry's tracer, span and meter, and what happened to the X-Ray SDKs on 25 February 2026?
7. List the AWS 7 Rs, and name the tool Microsoft now recommends instead of .NET Upgrade Assistant.

Lesson 12 in one line: .NET on AWS is the architecture you already run with different mechanics — Lambda finds a method by name and SnapStart or Native AOT answer its cold start, the SDK builds non-root, chiseled, multi-arch images without a Dockerfile, OpenTelemetry sits on APIs already in the base library, and a .NET Framework estate moves route by route after a 7 R triage.
After the track: build one real service end to end on your next project, then go deeper through the AWS Lambda .NET guide, Powertools for AWS Lambda (.NET), the .NET container reference, the ASP.NET Framework → ASP.NET Core migration guide and Aspire — and, if your team lives in Visual Studio, read the companion lesson 13 (Working in Visual Studio). Re-verify the dates in this PDF every November, starting with lesson 01 (The .NET Platform Map).